



Full Stack Developer - EHR Integration Intern Code Assessment

Clinical Data Reconciliation Engine (Mini Version)

Time Limit: 6 days

Submit: GitHub repository with code + README

Background Context

Healthcare providers often have conflicting information about the same patient across different systems. For example:

- **Hospital System A** says: Patient takes "Aspirin 81mg daily"
- **Clinic System B** says: Patient takes "Aspirin 325mg daily"
- **Pharmacy Records** show: Last filled "Aspirin 81mg" 2 days ago
- **Patient Portal** says: "Not currently taking aspirin"

Your task is to build a **mini clinical data reconciliation engine** that uses AI to determine the most likely accurate information.

Your Task

Build a full-stack application with the following components:

Part 1: Backend API (40%)

Create a REST API with these endpoints (you're limited to these endpoints):

1. POST `/api/reconcile/medication`

- Input: Array of conflicting medication records from different sources
- Output: Reconciled "most likely truth" with confidence score and reasoning



Example input:

JSON

```
{
  "patient_context": {
    "age": 67,
    "conditions": ["Type 2 Diabetes", "Hypertension"],
    "recent_labs": {"eGFR": 45}
  },
  "sources": [
    {
      "system": "Hospital EHR",
      "medication": "Metformin 1000mg twice daily",
      "last_updated": "2024-10-15",
      "source_reliability": "high"
    },
    {
      "system": "Primary Care",
      "medication": "Metformin 500mg twice daily",
      "last_updated": "2025-01-20",
      "source_reliability": "high"
    },
    {
      "system": "Pharmacy",
      "medication": "Metformin 1000mg daily",
      "last_filled": "2025-01-25",
      "source_reliability": "medium"
    }
  ]
}
```

Expected output:

JSON

```
{
  "reconciled_medication": "Metformin 500mg twice daily",
  "confidence_score": 0.88,
}
```



```
"reasoning": "Primary care record is most recent clinical encounter. Dose reduction appropriate given declining kidney function (eGFR 45). Pharmacy fill may reflect old prescription.",
"recommended_actions": [
  "Update Hospital EHR to 500mg twice daily",
  "Verify with pharmacist that correct dose is being filled"
],
"clinical_safety_check": "PASSED"
}
```

2. POST `/api/validate/data-quality`

- Input: Patient record with various fields
- Output: Data quality score (0-100) with breakdown by dimension

Example input:

JSON

```
{
  "demographics": {"name": "John Doe", "dob": "1955-03-15", "gender": "M"},
  "medications": ["Metformin 500mg", "Lisinopril 10mg"],
  "allergies": [],
  "conditions": ["Type 2 Diabetes"],
  "vital_signs": {"blood_pressure": "340/180", "heart_rate": 72},
  "last_updated": "2024-06-15"
}
```

Expected output:

JSON

```
{
  "overall_score": 62,
  "breakdown": {
    "completeness": 60,
    "accuracy": 50,
    "timeliness": 70,
    "clinical_plausibility": 40
  }
}
```



```
{
  "issues_detected": [
    {
      "field": "allergies",
      "issue": "No allergies documented - likely incomplete",
      "severity": "medium"
    },
    {
      "field": "vital_signs.blood_pressure",
      "issue": "Blood pressure 340/180 is physiologically implausible",
      "severity": "high"
    },
    {
      "field": "last_updated",
      "issue": "Data is 7+ months old",
      "severity": "medium"
    }
  ]
}
```

Part 2: AI Integration (35%)

Integrate an LLM API (OpenAI, Anthropic Claude, or similar) to:

- Generate clinical reasoning for reconciliation decisions
- Detect implausible data (e.g., impossible vital signs, drug-disease mismatches)
- Provide human-readable explanations

Requirements:

- Design effective prompts that include clinical context
- Handle API rate limits and errors gracefully
- Cache responses where appropriate to minimize API costs
- Document your prompt engineering approach

Part 3: Frontend Dashboard (25%)

Create a simple web interface that:

- Displays reconciliation results in a clinician-friendly format



- Shows data quality scores with visual indicators (red/yellow/green)
- Allows users to approve or reject AI suggestions
- Visualizes confidence scores and reasoning

Keep it simple - focus on clarity over design complexity.

Technical Requirements

Must Have:

- Clean, modular code architecture
- Input validation and error handling
- At least 5 unit tests covering core logic
- Basic authentication/API key protection
- README with setup instructions and design decisions

Tech Stack - Your Choice:

- Backend: Python
- Frontend: Tailwind CSS
- Database: Optional (can use in-memory storage)
- AI: OpenAI, Anthropic, or open-source LLM

Bonus Points (Optional):

- Confidence score calibration based on multiple factors
 - Duplicate record detection algorithm
 - Webhook support for real-time updates
 - Docker containerization
 - Simple deployment (Vercel, Railway, etc.)
-

Evaluation Criteria

Criterion	Weight	What We're Looking For
Code Quality	30%	Clean architecture, readable code, proper error handling



AI Integration	25%	Effective prompt design, smart use of LLM capabilities
Problem Solving	25%	Sound reconciliation logic, edge case handling
Product Thinking	20%	User-friendly interface, practical features, good documentation

Submission Guidelines

1. **GitHub Repository** with:

- All source code
- README with setup instructions
- Brief architecture decisions document (why you chose certain approaches)
- Any test data or examples

2. **README must include:**

- How to run the application locally
- Which LLM API you used and why
- Key design decisions and trade-offs
- What you'd improve with more time
- Estimated time spent

3. **Optional Video Demo** (2-3 minutes):

- Quick walkthrough of your solution
 - Highlight one technical decision you're proud of
-

Important Notes

- **No healthcare background required** - we'll explain any clinical concepts during review
 - **Don't overthink the clinical logic** - focus on building a solid technical foundation
 - **Ask questions** - email us if anything is unclear
 - **Time management** - we'd rather see a well-executed subset than rushed completeness
 - **Use libraries/frameworks** - don't reinvent the wheel
-

Test Data Provided



See the link below for data:

- [EHR Data](#)

Good luck! We're excited to see your approach.